Msc(DS&AI,CS)
IOT and Application

Teaching Scheme: Credit 01
Theory/Practical : (01 hrs./2 hrs.) / week

Examination Scheme:
Practical (PR): 50 Marks
Oral (OR): 50 Markss

Dr. Minaxi Doorwar
Assistant Professor in E&Tc Department
JSPM's ICOER, Wagholi, Pune

Course Objectives:

- lntroduction to loT, Overview of loT Building Blocks.
- Build small applications in loT for Engineering Applications using Sensors, Actuators, Microcontrollers and Cloud.
- Learn commonly used loT Simulation Hardware platforms.
- Understand different Communication Technologies used in loT.
- Development of application-level protocol and Security of loT Ecosystem.
- Understand loT applications in different domains.

Course Outcomes: On completion of the course, learner will be able to

CO1: Apply the knowledge of Applications/Devices, Protocols, and Communication Models of loT in practical scenarios.

CO2: Demonstrate the use of Sensors, Actuators, Microcontrollers, and Cloud platforms in developing small Mechanical Engineering loT-oriented applications.

CO3: Select and utilize commonly used loT Simulation Hardware platforms for designing and testing loT solutions.

CO4: Apply interfacing and communication technologies to develop functional loT systems.

CO5: lllustrate the process of loT application development and the integration of security measures within the loT ecosystem

CO6: Evaluate the applicability of present and future domain-specific loT applications in

real-world scenarios

Practical 1

The main objective of this experiment is,

A. Create an account on Wokwi

B. Identify the different devices and measuring instrument.

C. Simulate a simple circuit in Wokwi

Wokwi is a **free, web-based electronics simulator** for prototyping microcontrollers like **Arduino, ESP32, and Raspberry Pi Pico, allowing users** to build and test circuits with virtual **components (sensors, LEDs, displays) and** code directly in their browser before using physical hardware, offering safe experimentation, collaboration, and IoT simulation with built-in WiFi. It's great for learning, education (Wokwi Classroom), and IoT development without needing physical parts.

Key Features:

Online Simulation: Runs entirely in **the browser; no downloads needed to start**.

Broad Microcontroller Support: Simulates popular boards like **Arduino Uno, ESP32, STM32, and Raspberry Pi Pico**.

Rich Component Library: Includes **LEDs, buttons, sensors (IR, ultrasonic), motors, displays (OLED**), breadboards, and logic gates.

IoT Capabilities: Simulates **WiFi, MQTT, HTTP/HTTPS, and WebSockets** for IoT projects.

Code & Circuit Integration: **Write code (C++, MicroPython**) and wire components in an intuitive interface.

VS Code Integration: An **extension for Visual Studio Code** enables simulation directly within your development environment.

Collaboration & Sharing: Easily **share projects via links** for feedback and teamwork.

Wi-Fi Simulation: You can connect **simulated projects to the internet using protocols like MQTT, HTTP, and NTP**.

Debugging Tools: Includes a **virtual logic analyzer for capturing digital signals (I2C, SPI, UART) and advanced debugging with GDB.**

IDE Integration: Offers a **Wokwi for VS Code extension for local development and offline mode.**

Chips API: Advanced users can **create custom simulation models for new parts using C or WebAssembly**

Why Use Wokwi?

Risk-Free Learning: Experiment freely without damaging hardware.

Accessibility: Access unlimited parts and start instantly.

Code Validation: Helps differentiate between hardware and software issues.

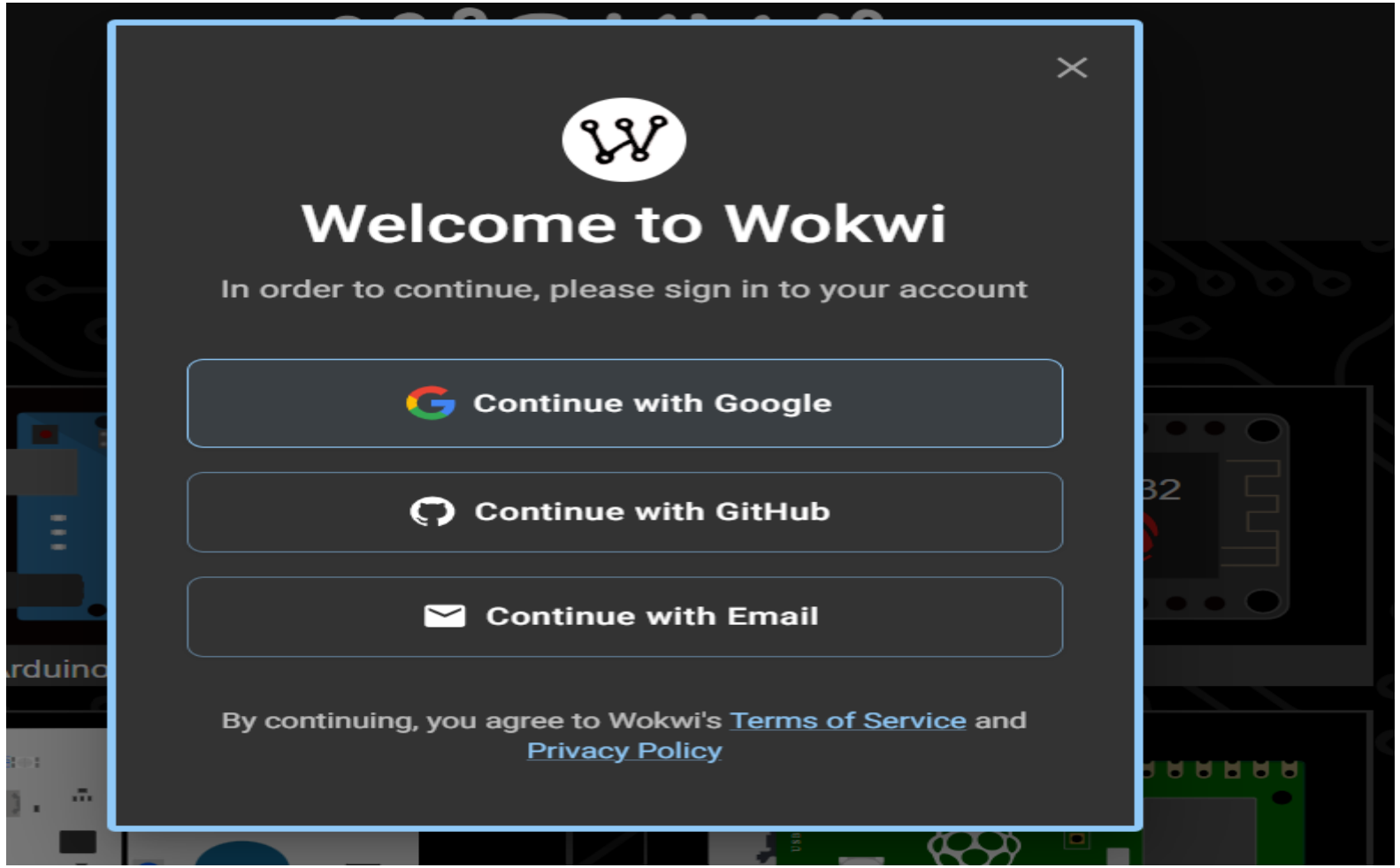
Real-World Application: Prototyped projects can often be deployed to real hardware.

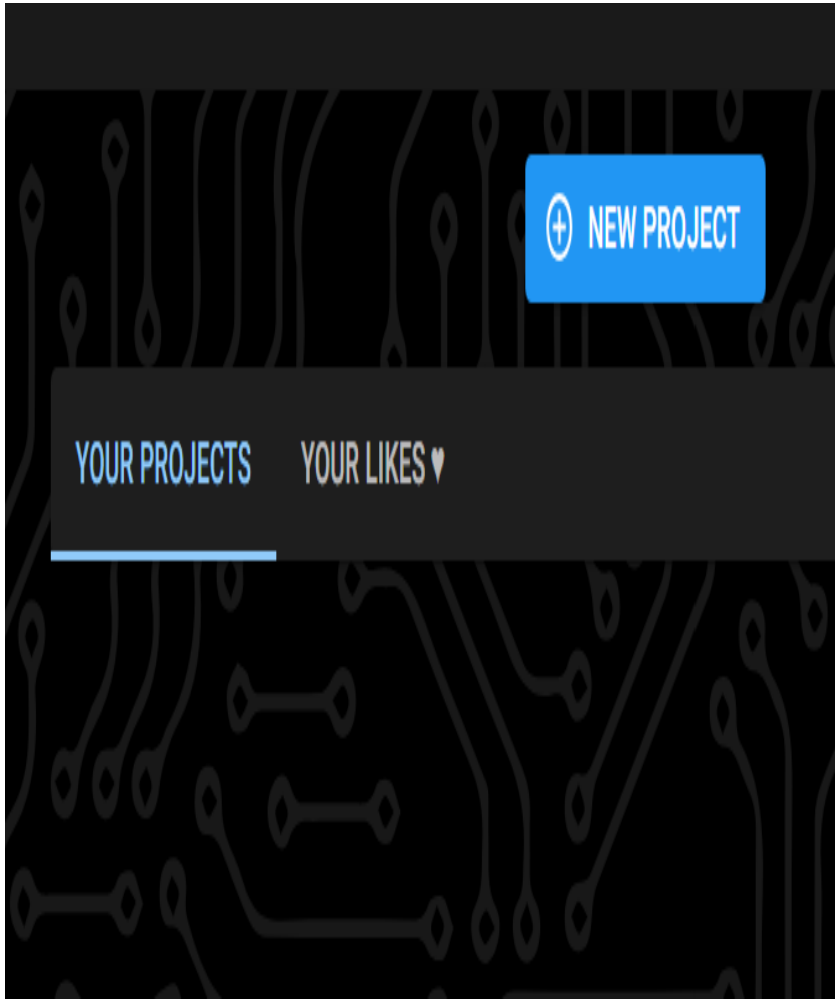
Arduino IDE (C++): Best for beginners due to its familiar interface, using C++ with the Arduino framework, which supports ESP32 features like Wi-Fi and Bluetooth.

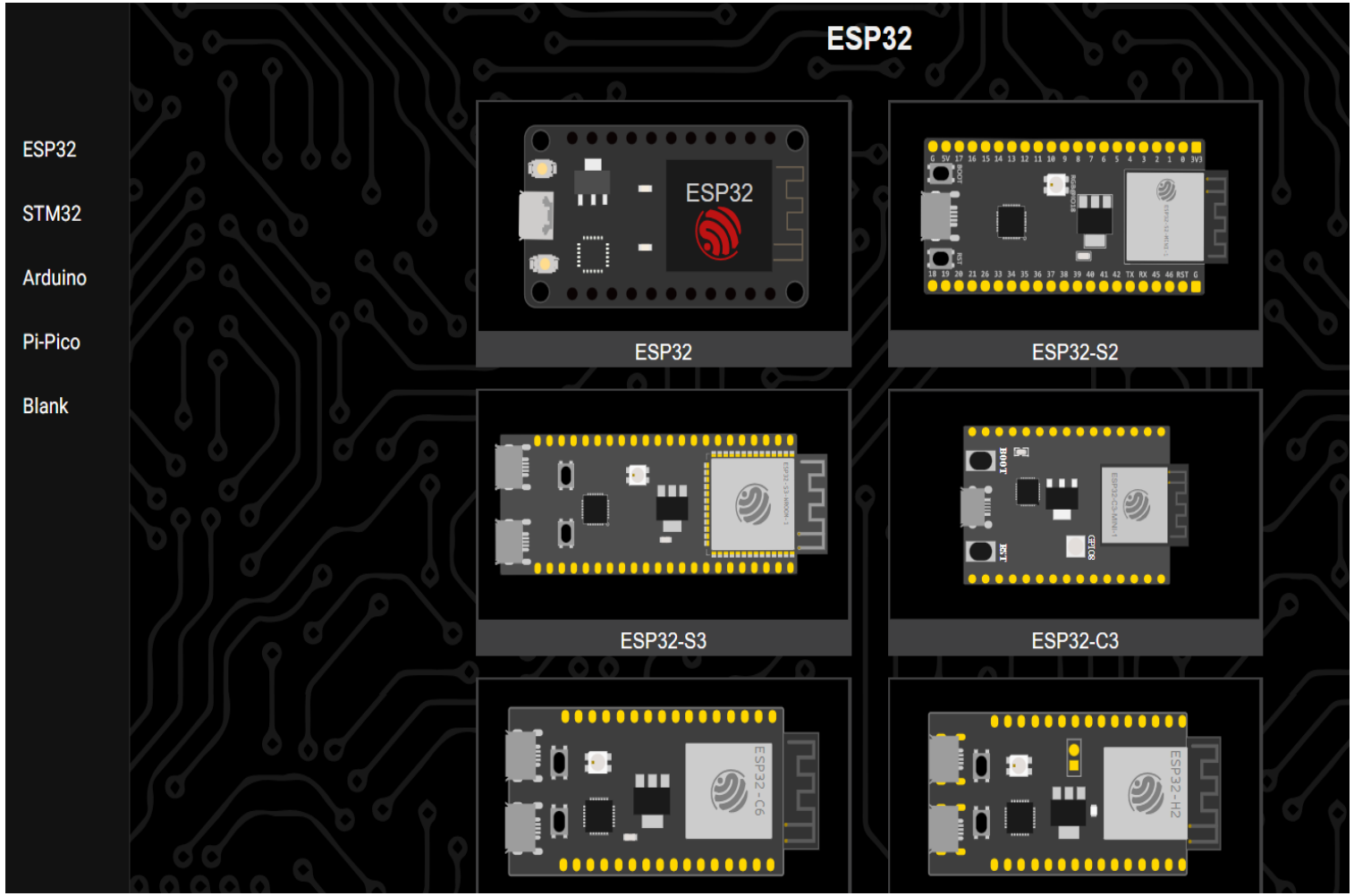
ESP-IDF (Espressif IoT Development Framework): Espressif's official framework for advanced C/C++ development, offering deep control over all hardware features.

MicroPython/CircuitPython: For those who prefer Python, offering a simpler, high-level programming experience on the ESP32.

WOKWi
World's most advanced ESP32 simulator
Discord Community
LinkedIn Group
Pricing
Simulate with Wokwi Online
uno
ARDUINO
Arduino (Uno, Mega, Nano)
ESP32
ESP32
STM32
STM32
Pi Pico


Welcome to Wokwi
In order to continue, please sign in to your account
Continue with Google
Continue with GitHub
Continue with Email
By continuing, you agree to Wokwi's Terms of Service and Privacy Policy


NEW PROJECT
YOUR PROJECTS
YOUR LIKES


ESP32
ESP32
STM32
Arduino
Pi-Pico
Blank
ESP32
ESP32-S2
ESP32-S3
ESP32-C3

ESP32 is a powerful microcontroller chip, while NodeMCU typically refers to a development board (often using the older ESP8266 chip, but also available with ESP32) designed for **easy prototyping with Wi-Fi and Bluetooth,** offering more features like dual cores, Bluetooth, and more peripherals than the original NodeMCU (ESP8266). **Think of ESP32 as the powerful brain, and NodeMCU as the user-friendly body (development board)** for it, with different NodeMCU versions housing different ESP Chip

ESP32 (The Chip)

Powerful Processor: Dual-core, faster clock speeds (up to 240MHz).

Connectivity: Built-in Wi-Fi & Bluetooth (BLE).

More Peripherals: More GPIO pins, better ADC, touch sensors, DAC, CAN bus, etc..

Memory: More SRAM and Flash memory.

NodeMCU (The Board)

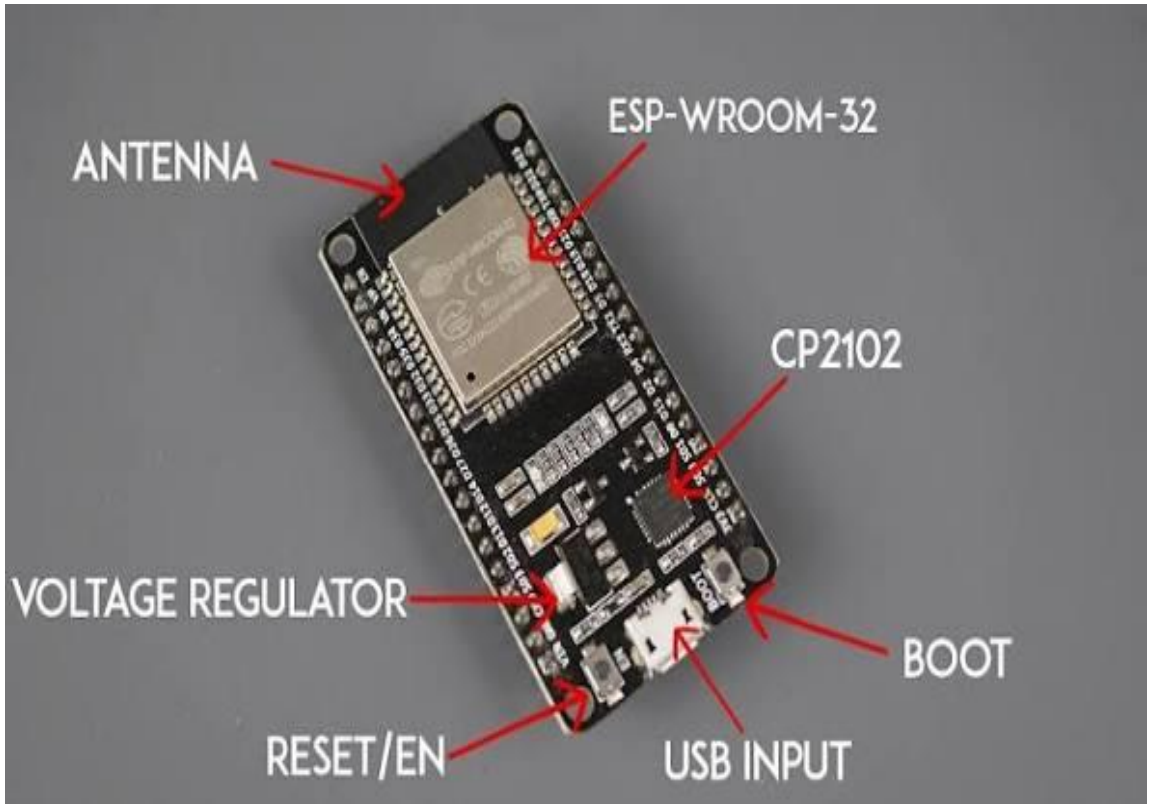
Development Board: A breakout board that makes the ESP chip easy to use.

Origin: Originally famous for the ESP8266 (cheaper, less powerful).

ESP32 Versions: Now available with the ESP32 chip (NodeMCU-32S, etc.).

Ease of Use: Breadboard-friendly, USB for power/programming, onboard regulators.

The ESP32 is a powerful and **versatile system-on-a-chip (SoC) microcontroller**, known for its integrated **Wi-Fi and Bluetooth capabilities.** This makes it a popular choice for **Internet of Things (IoT) applications, embedded systems, and other projects requiring wireless connectivity.**



The **ESP32 CP2102 refers to popular ESP32 development boards that use the Silicon Labs CP2102 chip as a USB-to-UART bridge**, allowing seamless **communication and programming (uploading code)** from a computer via USB. These boards feature the powerful dual-core ESP32 microcontroller with built-in **Wi-Fi and Bluetooth, making them ideal for IoT projects, home automation, and robotics, with common types like NodeMCU using this chip for easy setup and development**. You usually need to install the CP2102 driver on your PC for the board to be recognized.

ESP32 Microcontroller: The brain, offering dual-core processing, Wi-Fi (2.4GHz), and Bluetooth (Classic & BLE).

CP2102 Chip: Converts USB signals to serial (UART) for the ESP32, acting as the **communication interface**.

USB Interface: Often Micro-USB or Type-C, connecting to your PC.

GPIO Pins: Numerous pins for connecting sensors, LEDs, and other components.

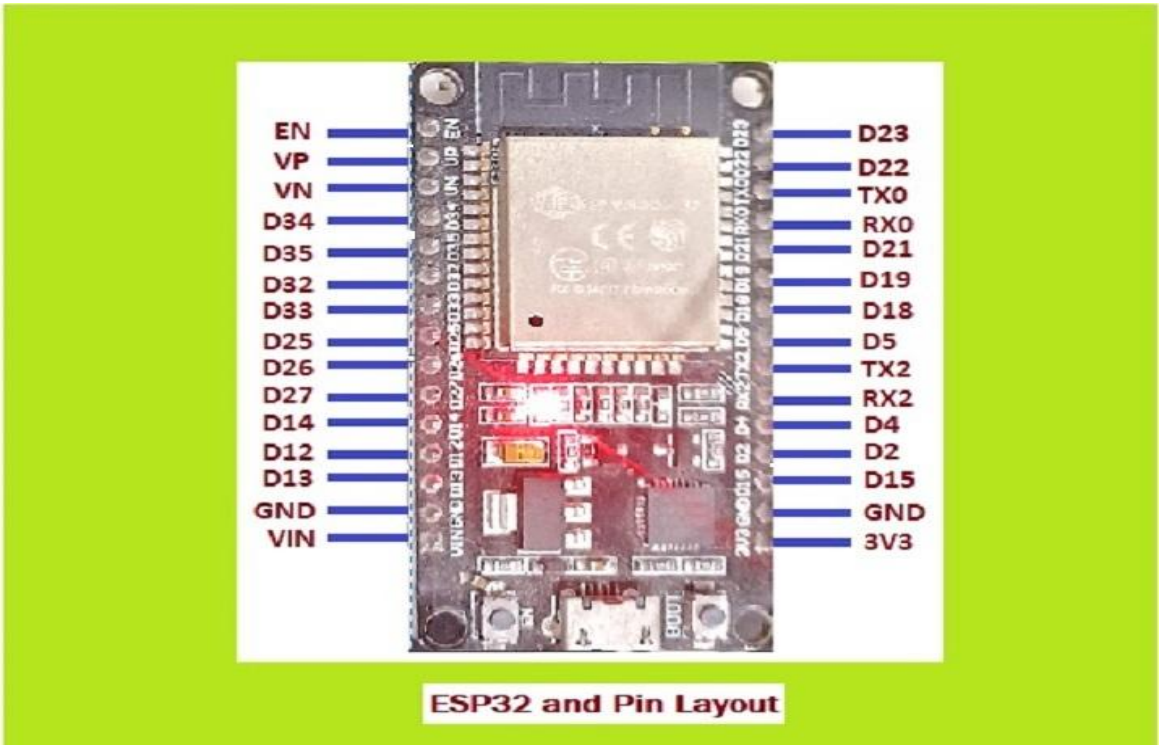
Memory: Typically 4MB Flash and 520KB SRAM.

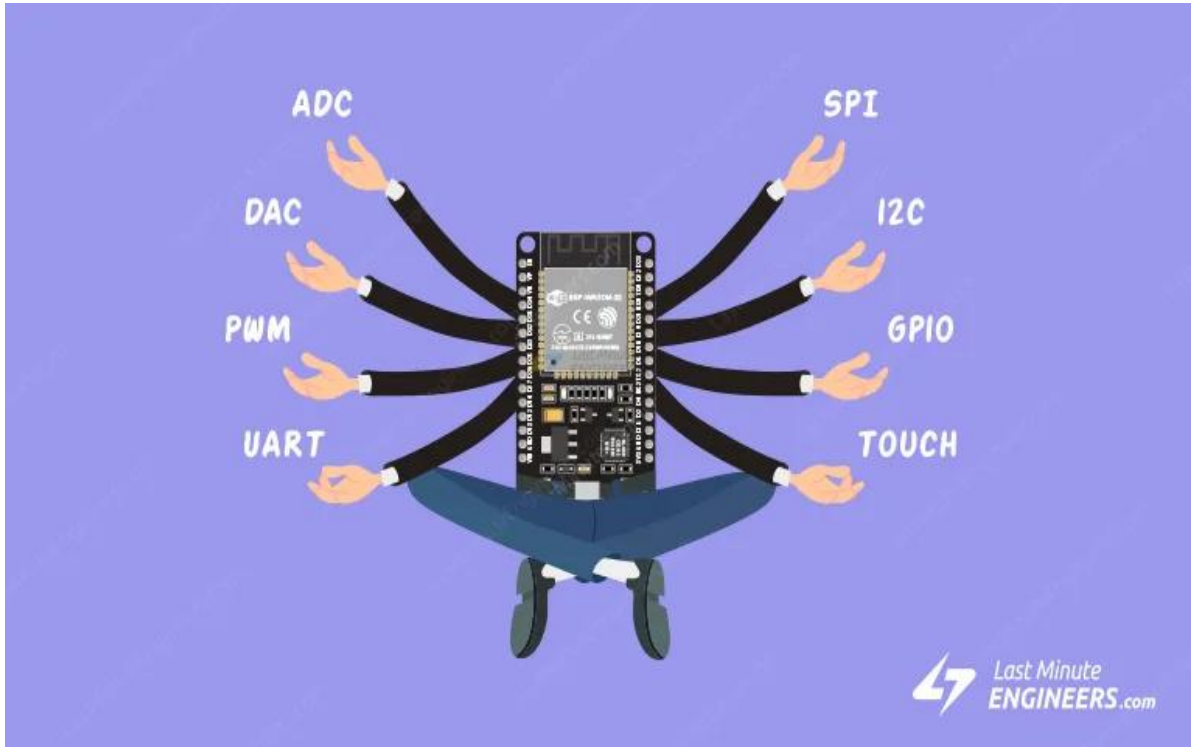
Common Boards:

NodeMCU ESP32: A popular form factor for these boards, often featuring the CP2102.

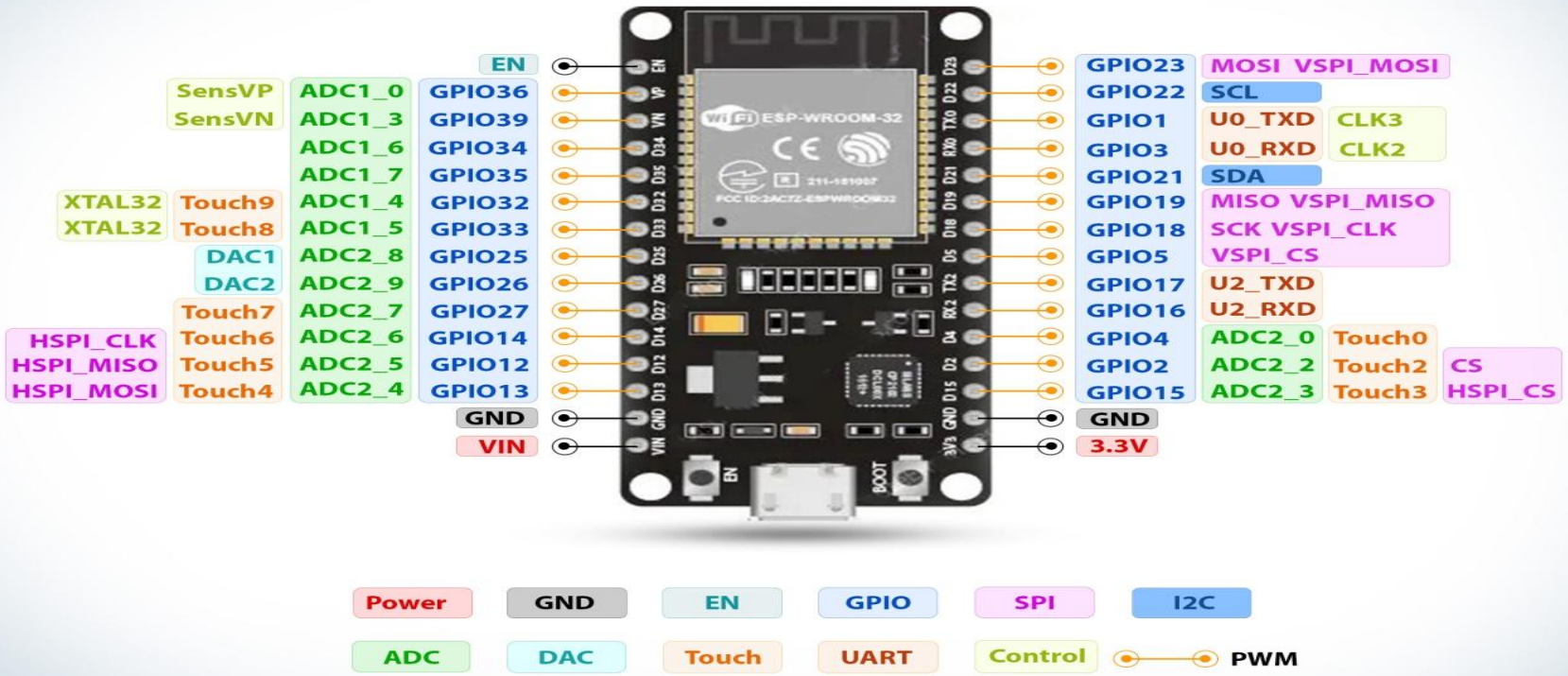
ESP32 DevKit: Various versions use the CP2102 as the USB bridge

ESP32 has 48 GPIO pins in total, only 25 of them are broken out to the pin headers on both sides of the development board. These pins can be assigned a variety of peripheral duties, including: **30-pin ESP32 devkit v1** development board.





ESP32 DEV. BOARD PINOUT



ESP32 pin configuration **involves setting GPIO pins for various functions (digital I/O, ADC, DAC, PWM, UART, SPI, I2C)** using code (like Arduino's pinMode()), but special attention is needed for **Strapping Pins** (GPIO 0, 2, 12, 15) and **Flash Pins** (GPIO 6-11), which have critical boot-time states, and unused pins like 34-39.

Key Pin Types & Functions

- **GPIO (General Purpose Input/Output):** Most flexible pins for digital input/output, PWM, etc..
- **ADC (Analog-to-Digital Converter):** Read analog voltages (e.g., sensors); ESP32 has ADC1 (GPIO 32-39) and ADC2 (GPIO 0, 2, 4, 12-15, 25-27).
- **DAC (Digital-to-Analog Converter):** Output analog voltages (GPIO 25, 26).
- **PWM (Pulse Width Modulation):** For dimming LEDs, controlling motors (many GPIOs support this).
- **Serial (UART, SPI, I2C):** For communication; often multiple channels (e.g., VSPI/HSPI for SPI). are input-only. Always check the specific board's pinout and use libraries for complex peripherals to handle multiplexing and pin assignments easily.

.

Critical & Special Pins to Know

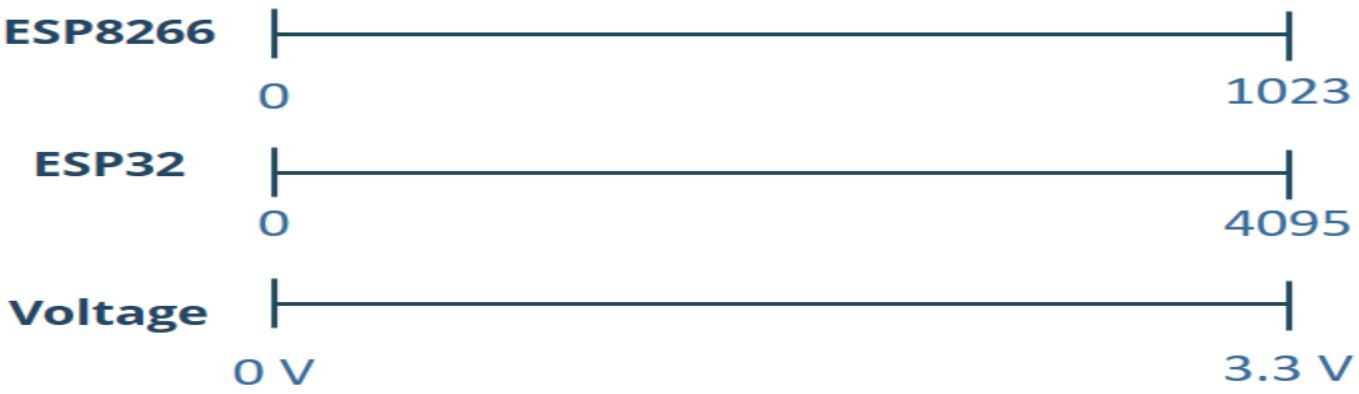
- **Strapping Pins (Boot Mode):** GPIO 0, 2, 12, 15 must be in specific states (HIGH/LOW) during boot for flashing or normal operation; avoid connecting to VCC/GND directly if possible.
- **Flash SPI Pins:** GPIO 6-11 are used for internal flash memory and should never be used for other devices.
- **Input-Only Pins:** GPIO 34, 35, 36, 39 can only be used as inputs.
- **UART0 (Serial):** GPIO 1 (TX0) and GPIO 3 (RX0) are used for programming/debugging via USB

The Power Pins of ESP32

ESP32 has 2 GND output pins. It has two pins for positive voltages, **Vin and 3V3.** Vin can be connected to an external voltage source of 5V to 14V. 3V3 is the pin that can output 3.3 volts and powers the ESP32.

General Purpose Input Output (GPIO) Pins of ESP32

There are **25 GPIO pins in ESP32 wroom 30 pin** microcontroller. Although these can be used for different functions, there are some unsafe pins and these should be avoided, for example, RX0 and TX0.



GPIO Pins	ESP32 Pins
GPIO 1	TX0
GPIO 2	D2
GPIO 3	RX0
GPIO 4	D4
GPIO 5	D5
GPIO 12	D12
GPIO 13	D13
GPIO 14	D14
GPIO 15	D15
GPIO 16	RX2
GPIO 17	TX2
GPIO 18	D18
GPIO 19	D19
GPIO 21	D21
GPIO 22	D22
GPIO 23	D23
GPIO 25	D25
GPIO 26	D26
GPIO 27	D27
GPIO 32	D32
GPIO 33	D33
GPIO 34	D34
GPIO 35	D35
GPIO 36	VP
GPIO 39	VN

Analog Pins of ESP32:

SP32 has 15 analog-enabled pins called ADC pins. The Analog to Digital converters ADC1 and ADC2 of ESP32 can be used to map the voltages from 0V to 3.3 V to numbers between 0 to 4096. **The ESP32 has 18 x 12 bits ADC input channels (while the ESP8266 only has 1x 10 bits ADC).**

I2C (Inter-Integrated Circuit) Interface of ESP32:

I2C interface of ESP32 is used for communications between **ESP32 (Master) and sensors (Slaves)**. For such communications, the ESP32's D22 and D21 pins are allocated.

GPIO 21 (SDA):Serial Data

GPIO 22 (SCL):Serial Clock

Analog Pins	ESP32 Pins
ADC1_0	VP
ADC1_3	VN
ADC1_4	D32
ADC1_5	D33
ADC1_6	D34
ADC1_7	D35
ADC1_8	D25
ADC1_9	D26
ADC2_0	D4
ADC2_2	D2
ADC2_3	D15
ADC2_4	D13
ADC2_5	D12
ADC2_6	D14
ADC2_7	D27

DAC Pins of ESP32:

DAC Pins	ESP32 Pins
DAC1	D25
DAC2	D26

Touch Pins of ESP32

There are 9 touch pins in ESP32 wroom 30 pin microcontroller. They are listed from Touch 0 to T9 excluding T1 which is not there in the 30-pin microcontroller. These can sense variations in anything **that holds an electrical charge, like the human skin.** So they can **detect variations induced when touching the GPIOs with a finger**. These pins can be **easily integrated into capacitive pads and replace mechanical buttons.**

Touch Pins	ESP32 Pins
Touch 0	D4
Touch 2	D2
Touch 3	D15
Touch 4	D13
Touch 5	D12
Touch 6	D14
Touch 7	D27
Touch 8	D33
Touch 9	D32

Transmitter and Receiver Pins of ESP32:

Although ESP32 has three UART (Universal Asynchronous Receiver Transmitter) interfaces, only the following pins of UART2 are safe options to use while using the devices like distance sensors.

Transmitter/ Receiver Pins	ESP32 Pins
TX2	D22
RX2	D21

Input only pins

GPIOs 34 to 39 are GPIs – input only pins. These pins don't have internal pull-up or pull-down resistors. They can't be used as outputs, so use these pins only as inputs:

- GPIO 34
- GPIO 35
- GPIO 36
- GPIO 39

PWM

The ESP32 **LED PWM controller has 16 independent channels** that can be configured to generate PWM signals with different properties. All pins that can act as outputs can be used as PWM pins (GPIOs 34 to 39 can't generate PWM).

To set a PWM signal, you need to define these parameters in the code:

- **Signal's frequency, Duty cycle, PWM channel, GPIO where you want to output the signal.**

SPI flash integrated on the ESP-WROOM-32

GPIO 6 to GPIO 11 are exposed in some ESP32 development boards. However, these pins are connected to the integrated SPI flash on the ESP-WROOM-32 chip and are not recommended for other uses. So, don't use these pins in your projects:

- GPIO 6 (SCK/CLK)
- GPIO 7 (SDO/SD0)
- GPIO 8 (SDI/SD1)
- GPIO 9 (SHD/SD2)
- GPIO 10 (SWP/SD3)
- GPIO 11 (CSC/CMD)

Processing and memory

- **Dual-core or single-core processor:** The standard ESP32 features a dual-core, 32-bit Tensilica Xtensa LX6 microprocessor, with some variants like the ESP32-S0WD using a single-core version. Newer ESP32-C series models use a single-core RISC-V CPU, and ESP32-S3 uses a dual-core Xtensa LX7 CPU with machine learning acceleration.
- **Adjustable clock frequency:** The clock speed is adjustable from **80 MHz to 240 MHz**, allowing developers to balance performance and power consumption.
- **Ultra-low-power (ULP) co-processor**: For **extended battery life**, the ULP co-processor can perform simple tasks like sensor monitoring while the main CPU is in deep sleep.
- **Internal memory**: Memory includes **520 KB of on-chip SRAM** for data and instructions, plus additional ROM and RTC SRAM for boot and low-power operations.
- **Expandable flash memory:** Modules typically come with **4 MB of external flash memory** for storing programs and data, with some variants supporting additional external PSRAM.

Wireless connectivity

- **Wi-Fi: Supports 802.11 b/g/n Wi-Fi**, allowing the ESP32 to connect to routers in Station mode, create its own network in Access Point (SoftAP) mode, or operate in both simultaneously.
- **Bluetooth:** Features dual-mode Bluetooth connectivity with support for both Classic Bluetooth (BR/EDR) and Bluetooth Low Energy (BLE), enabling it to communicate with a wide range of devices.
- **Wi-Fi 6 (802.11ax):** Newer variants, such as the ESP32-C6, also feature Wi-Fi 6 support

Power management

- **Multiple low-power modes:** The ESP32 is designed for energy efficiency, offering several power-saving modes to optimize battery life.
- **Modem-sleep**: Wi-Fi and Bluetooth are disabled, but the CPU remains active. Power consumption is ~3–20 mA.
- **Light-sleep:** The CPU and most RAM are clock-gated, but the Wi-Fi/Bluetooth connection is maintained. Current consumption is approximately 0.8 mA.
- **Deep-sleep**: Powers down the CPU and most RAM, with only the real-time clock (RTC) and ULP co-processor remaining active. This dramatically reduces power consumption, making it ideal for battery-operated devices.
- **Hibernation**: The most extreme power-saving mode, where nearly all hardware is turned off, with consumption as low as 5 µA.

Peripherals and interfaces

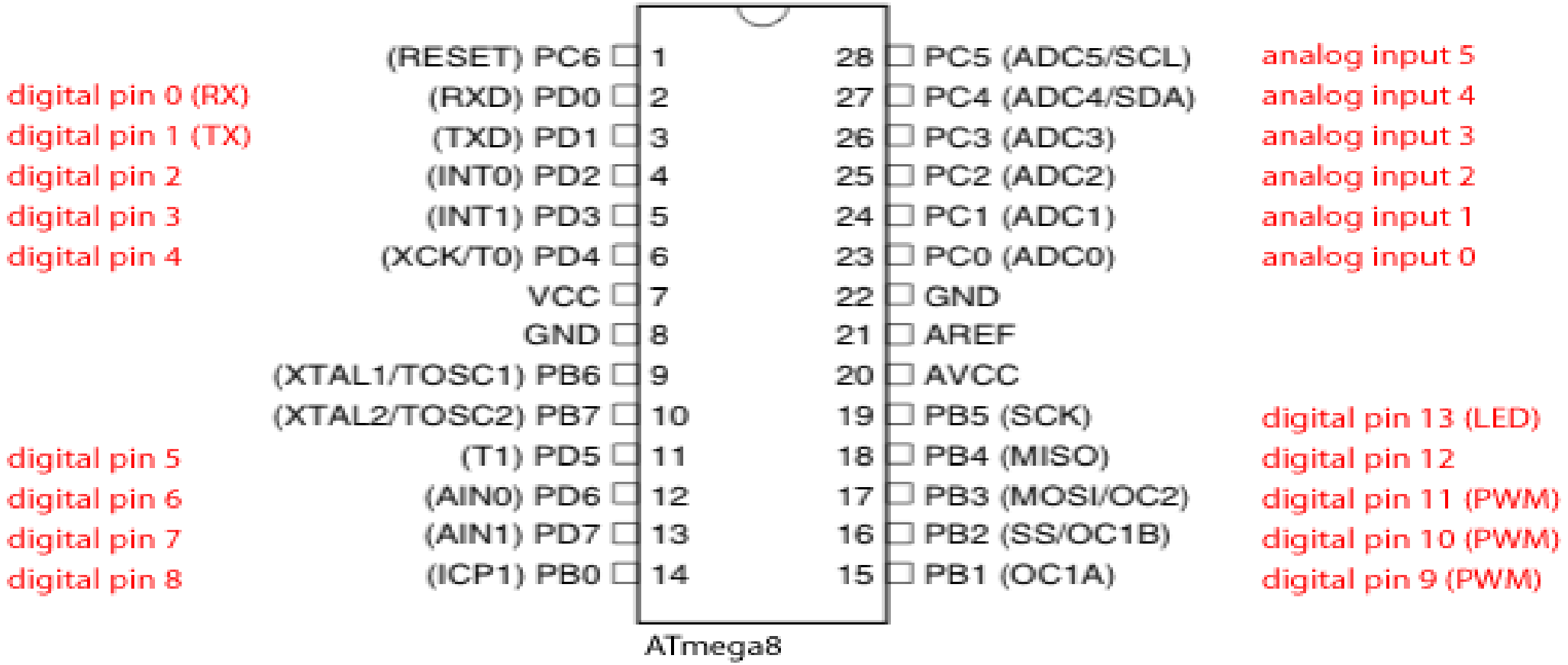
- **Extensive GPIOs:** Offers numerous programmable General-Purpose Input/Output (GPIO) pins for connecting to a wide variety of sensors and peripherals.
- **Analog-to-Digital Converter (ADC)**: Features multiple 12-bit SAR ADC channels for reading analog sensor data.
- **Digital-to-Analog Converter (DAC):** Includes 8-bit DAC channels for converting digital signals to analog voltages.
- **Capacitive touch sensors**: Contains built-in touch sensors, which can be used to create touch-sensitive pads without external hardware.
- **Common communication protocols**: Supports standard interfaces such as SPI, I^2C, I^2S, UART, and PWM, allowing for flexible connectivity.
- **Other peripherals**: Depending on the model, additional peripherals can include a Hall sensor, Ethernet MAC interface, and an SD/SDIO/MMC Host controller

Security

- **Secure boot:** Ensures that only trusted and authenticated software can execute on the device, preventing untrustworthy code from running.
- **Flash encryption:** Encrypts the contents of the external flash memory, protecting intellectual property and sensitive data.
- **Hardware cryptographic acceleration:** Features hardware acceleration for cryptographic algorithms like AES, SHA-2, and RSA, ensuring secure communication.
- **Secure OTA updates**: Allows for secure Over-the-Air firmware updates, ensuring that new code is authenticated and encrypted during transit.

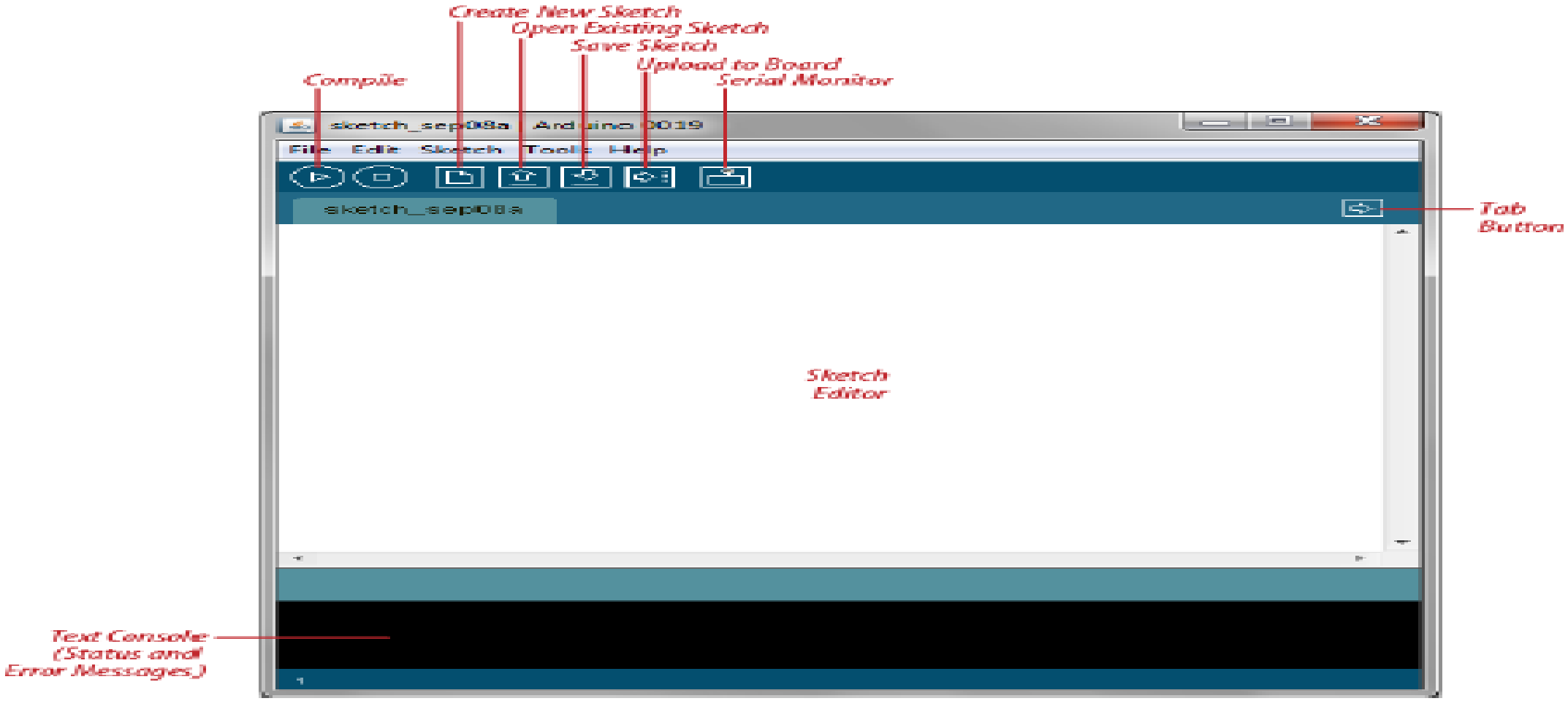
Arduino Pin Mapping

www.arduino.cc



What do we need?

Arduino IDE


Create New Sketch
Open Existing Sketch
Save Sketch
Upload to Board
Serial Monitor
Compile
sketch_sep08a | Arduino 0019
File Edit Sketch Tools Help
sketch_sep08a
Tab Button
Sketch Editor
Text Console (Status and Error Messages)

Arduino Programming Language

Control Statements	Operators	Functions
if if..else for while do..while switch..case	+, -, *, /, <, >, <=, >=, <>, &&, ||, !	pinMode() digitalWrite() digitalRead() delay() analogRead() analogWrite()

Important functions

- Serial.println(value);
  - Prints the value to the Serial Monitor on your computer
- pinMode(pin, mode);
  - Configures a digital pin to read (input) or write (output) a digital value
- digitalRead(pin);
  - Reads a digital value (HIGH or LOW) on a pin set for input
- digitalWrite(pin, value);
  - Writes the digital value (HIGH or LOW) to a pin set for output

- Delays are essential in embedded systems, unlike high-performance systems where we want the program to execute as fast as possible
- Delays are used to synchronize events, or read inputs with a specific sampling freqency (more on Bus/Wait I/O)

Arduino example:

```
delay(int milliseconds)
//creates a delay in ms
delayMicroseconds(int microseconds)
//creates a delay in µs

delay(1000); //one second delay
delayMicroseconds(10); //10 µs delay
```

General Input/Output

Embedded processors receive input from their environment (sensors) and produce output that change that environment (actuators) or give information (indicators)

These require reading and writing to single or multiple bit I/O ports

sketch_may14e | Arduino 1.0.5

File Edit Sketch Tools Help

sketch_may14e §

```
void setup() {
}



void loop() {
}
```

www.TheEngineeringProjects.com

8 Arduino Uno on COM1

1.Install Arduino IDE: Download and install from Arduino.

2.Add ESP32 Board URL: Go to File > Preferences, add https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_dev_index.json to "Additional Board Manager URLs".

3.Install ESP32 Board Package: Open Tools > Board > Boards Manager, search for "ESP32", and install the one from Espressif Systems.

4.Connect & Select Board: Plug in your ESP32, go to Tools > Port to select the COM port, and Tools > Board to choose your specific ESP32 model (e.g., "ESP32 Dev Module").

5.Upload Code: Write your code (e.g., a simple blink sketch using LED_BUILTIN or pin 2) and click upload. You might need to press the board's **BOOT** button briefly when the IDE says "Connecting...".

First program (Blink LED)

Most ESP32 boards have an onboard LED on **GPIO 2**.

```
#define LED_PIN 2

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  delay(1000);
}
```

```
void setup() {
  Serial.begin(115200);
  Serial.println("ESP32 Started!");
}

void loop() {
  Serial.println("Hello ESP32");
  delay(1000);
}
```

```
void setup() {
  pinMode(2, OUTPUT);       // Set GPIO 2 as output
  Serial.begin(115200);     // Start serial communication
}
```

Think of setup() as preparation before work starts.

loop() function

Purpose:Runs again and again forever

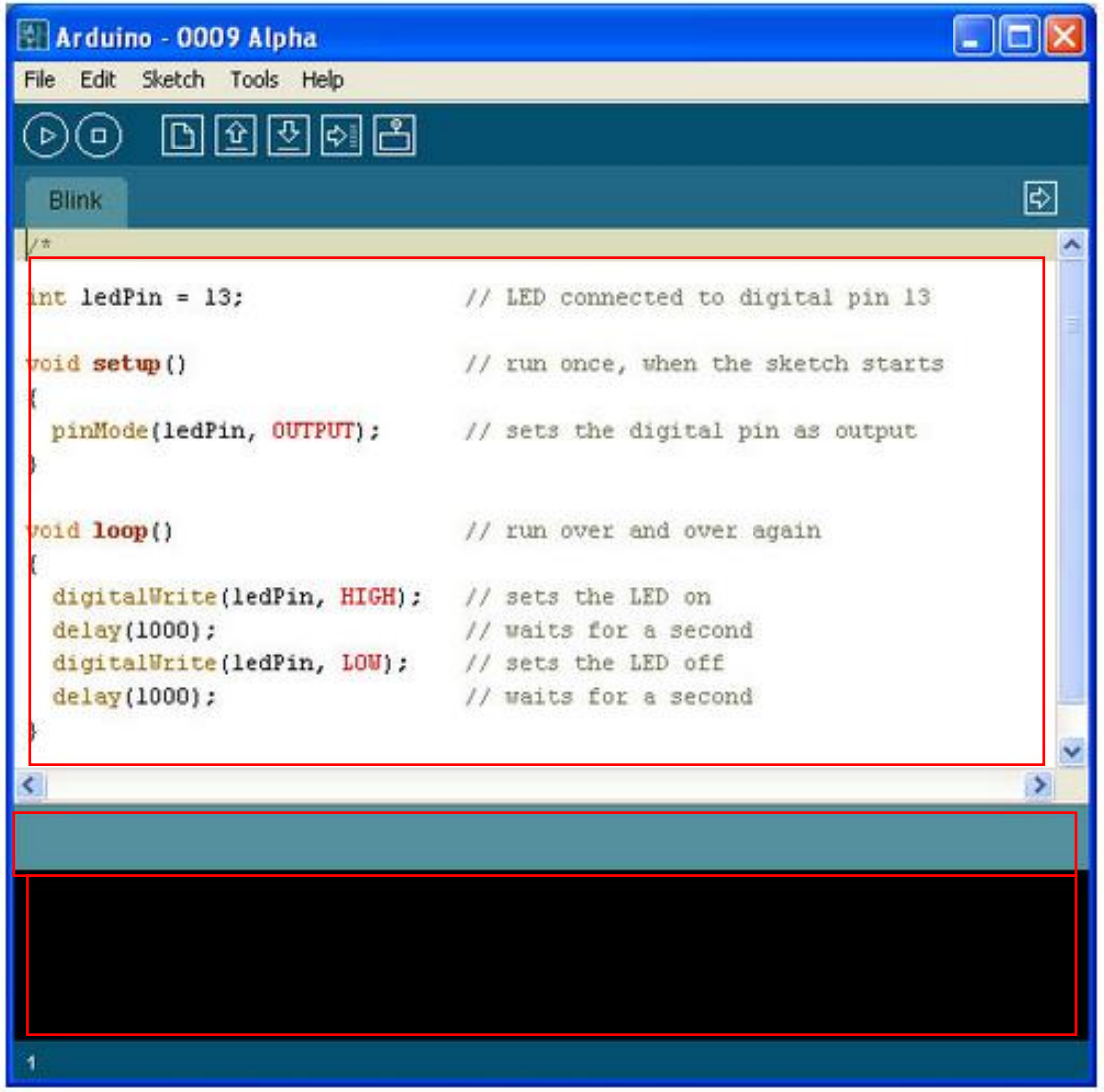
Examples (Arduino)

```
void loop (){
digitalWrite(13,HIGH);
delay(500);
digitalWrite(13, LOW);
}
```

This creates an infinite loop without the programmer using a goto (which is bad programming practice)

These turn pin 77 on and off (assuming there is something like a LED there without using logic instructions and I/O ports

This creates a half-second (500 ms) delay without directly accessing the timers



Input

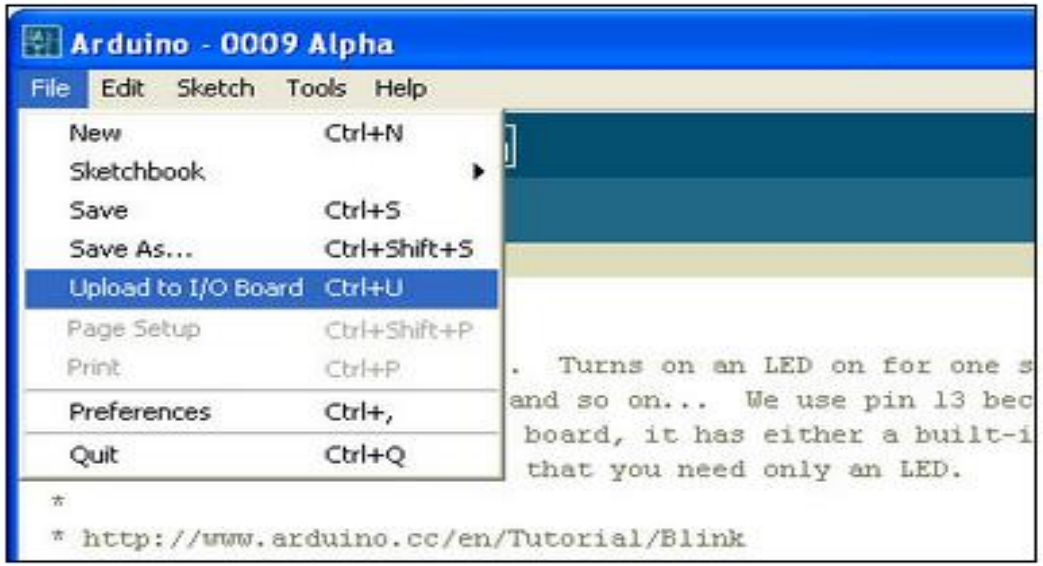
Program Notification Area

Status Bar

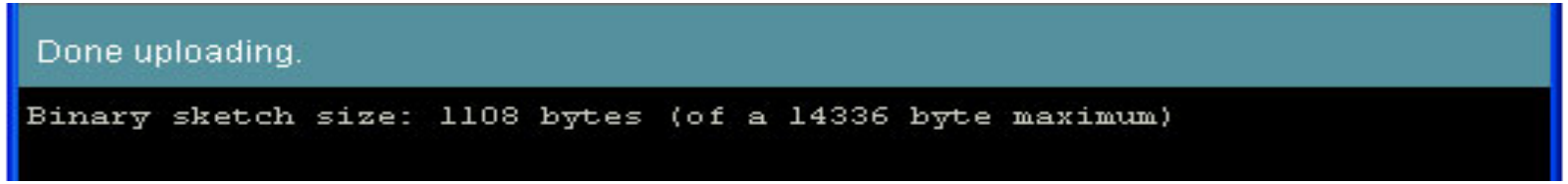
- 5.- Verify/Compile

- 6.- Once it compiles, you must see the following messages in the **Status bar** and the **Program notification Area**

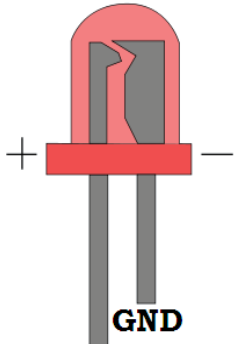
- 7.- Upload it



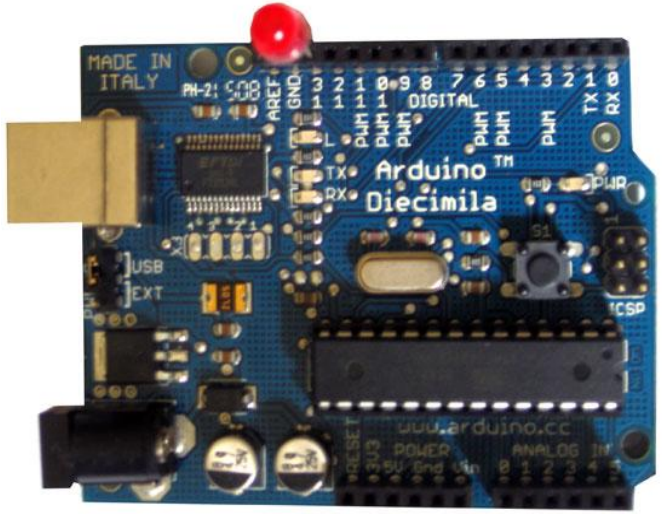
- 8.- Once it upload, you must see the following messages in the **Status bar** and the **Program notification Area**



- 9.- Insert LED

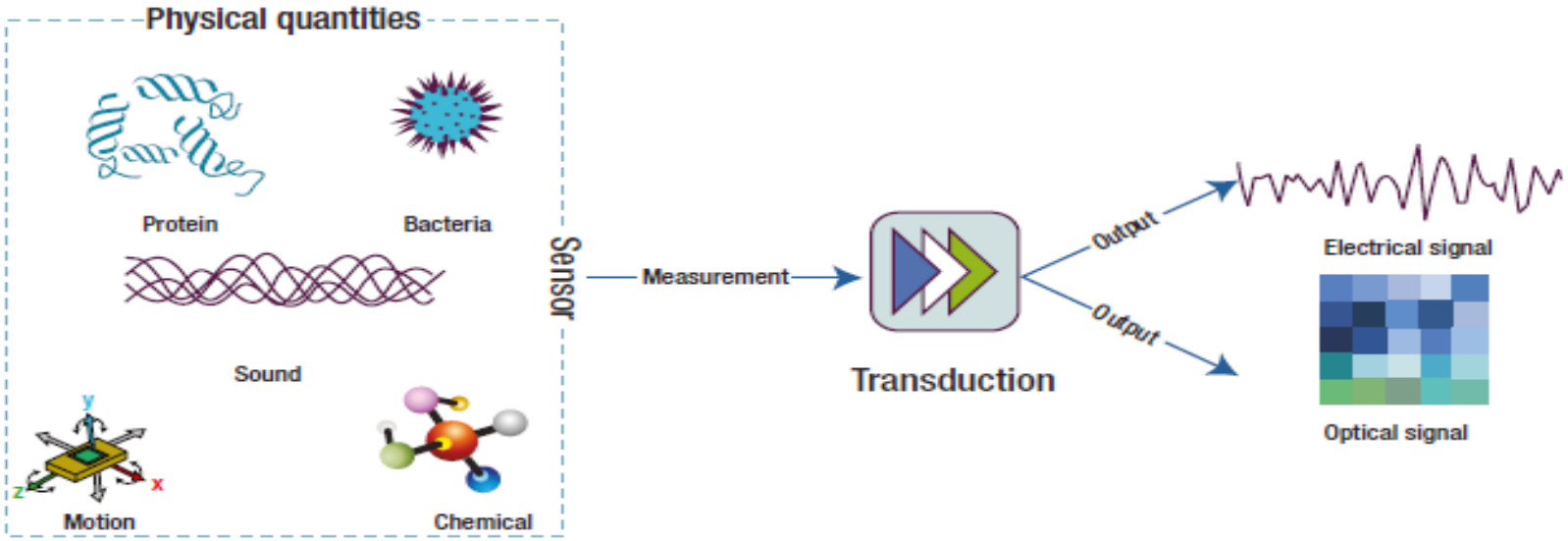


- It is blinking!!!



What is a sensor?

- A device that receives a stimulus and responds with an electrical signal.
- A special type of transducer (device that converts one type of energy into another)
- A sensor is a device that **detects physical, chemical, or biological** changes in the environment (such as **heat, light, sound, pressure, or motion**) and **converts them into signals, typically electrical, that can be measured, recorded, or used to trigger responses**

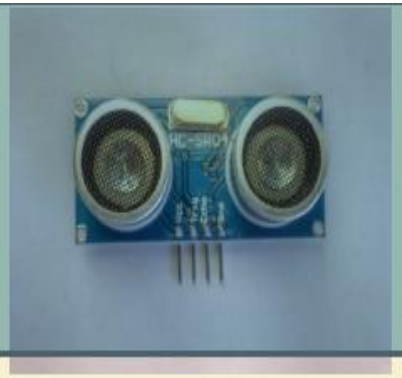


- **Functionality:** Sensors monitor, measure, and control parameters, providing data to processors like computers or microcontrollers.
- **Components:** A sensor typically includes a sensing element (detects input), a transducer (converts to a signal), and signal conditioning units (amplifies/processes the signal)
- Sensors can be categorized as **active (generate their own signal) or passive (require external power).**

Sensor Types

Light	• Light Dependent resistor • Photo-diode
Temperature	• Thermocouple • Thermistor
Force	• Strain gauge • Pressure switch
Position	• Potentiometer, Encoders • Opto-coupler
Speed	• Reflective/ Opto-coupler • Doppler effect sensor
Sound	• Carbon Microphone • Piezoelectric Crystal
Chemical	• Liquid Chemical sensor • Gaseous chemical sensor

Pressure Sensor
Source: Wikimedia Commons

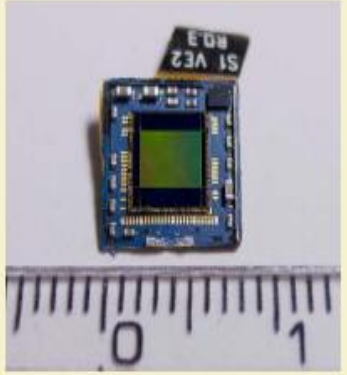


Ultrasonic Distance Sensor
Source: Wikimedia Commons

Tilt Sensor
Source: Wikimedia Commons

Infrared Motion Sensor
Source: Wikimedia Commons

Analog Temperature Sensor
Source: Wikimedia Commons



Camera Sensor
Source: Wikimedia Commons

An actuator is a component responsible for **moving or controlling a mechanism by converting energy (electrical, pneumatic, or hydraulic) into mechanical force.** Often called a "mover," it is crucial for automating systems in manufacturing, automotive, and robotics to perform actions like lifting, pushing, or tightening.

Actuator

- ✓ An actuator is a component of a machine or system that moves or controls the mechanism or the system.
- ✓ An actuator is the mechanism by which a control system acts upon an environment
- ✓ An actuator requires a control signal and a source of energy.

Function: Translates control signals (low energy) into mechanical motion (high energy).

Types of Motion: Actuators can produce linear motion (straight-line, such as pushing/pulling) or rotary motion (rotating).

Types of Energy Sources:

Electric: Uses motors (AC/DC) for precise, clean operation.

Hydraulic: Uses pressurized fluids to generate high-force, heavy-duty movement.

Pneumatic: Uses compressed air for fast, repetitive, or simple movements.

Actuator Types

- Hydraulic
- Pneumatic
- Electrical
- Thermal/ Magnetic
- Mechanical

Types of Actuators

Actuators are broadly classified by their power source and the type of motion they produce (linear or rotary).

Electric Actuators: These use electrical energy, often via a motor or solenoid, to generate motion.

Advantages: Offer high precision positioning, are generally cleaner and quieter, and easily integrate with control systems.

Applications: Used widely in robotics, medical devices, factory automation, and home automation systems like automatic doors and adjustable furniture.

Hydraulic Actuators: These use pressurized fluid (liquid) to generate motion.

Advantages: Capable of generating very large forces and high torque even with a compact design due to the incompressibility of liquids.

Applications: Commonly found in heavy-duty machinery like excavators, hydraulic

Pneumatic Actuators: These use compressed air or gas pressure to create movement.

Advantages: A low-cost option, simple, reliable, and suitable for high-temperature or explosive environments where electricity or hydraulic fluid might be a hazard.

Applications: Used in robotics, packaging lines, material handling systems, and controlling industrial valves.

Mechanical Actuators: These use physical mechanisms such as gears, cams, or screws to create motion,

Advantages: Simple in design, durable, and low-cost.

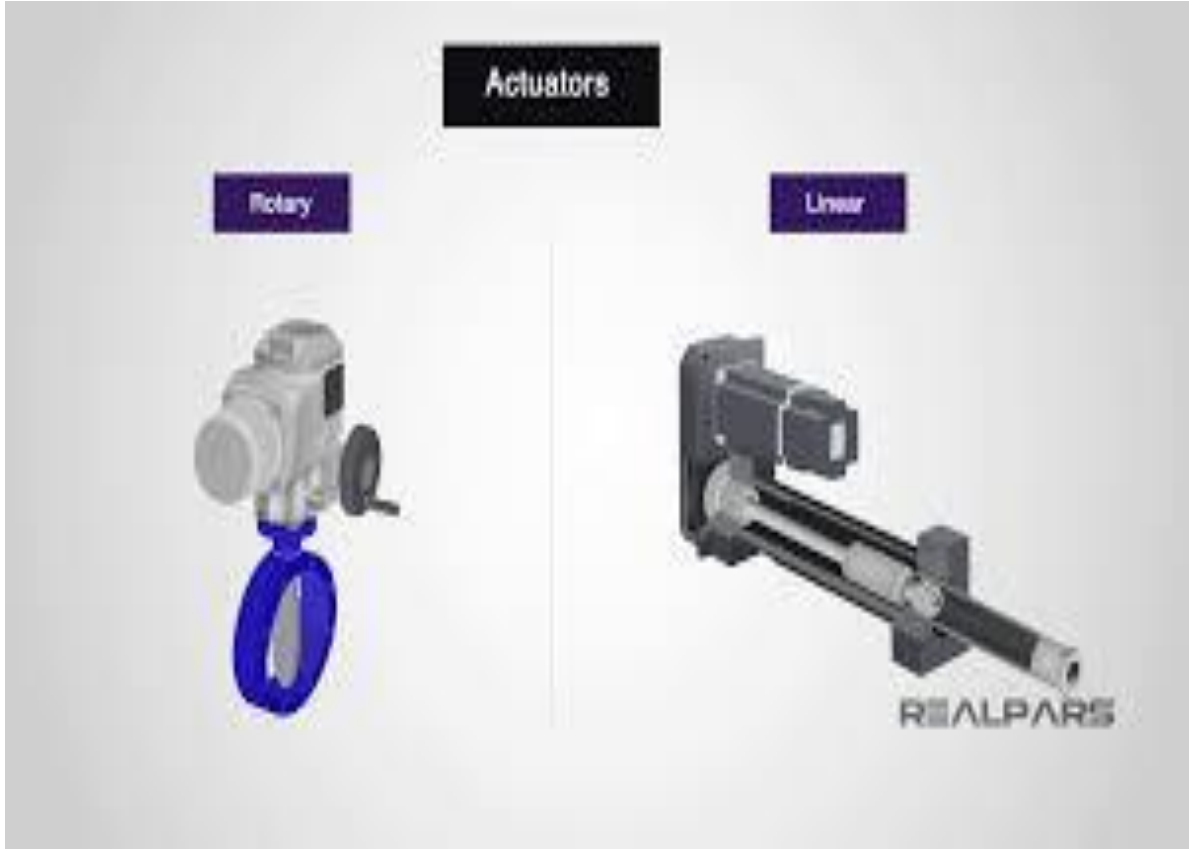
Applications: Examples include a car's crankshaft or a manual hand-crank machine.

Thermal/Magnetic Actuators: These utilize temperature changes (material expansion/contraction) or magnetic fields to produce motion.

Advantages: Used in miniaturized applications (MEMS) or for passive, energy-efficient temperature regulation.

Applications: Used in thermostats and certain types of electric switches or safety shutoff processes based on

DC Motor



```
#define LED 2

void setup() {
  pinMode(LED, OUTPUT);     // Set LED pin
}

void loop() {
  digitalWrite(LED, HIGH);  // LED ON
  delay(500);
  digitalWrite(LED, LOW);   // LED OFF
  delay(500);
}
```

Creating infinite loops

```
void loop()  //blink a LED repeatedly
{
digitalWrite(4,HIGH); // turn the LED on
delay(500); // wait 500 milliseconds
digitalWrite(4,LOW); // turn the LED off
delay(500); // wait 500 milliseconds
}
```

```
#define LED1 2
#define LED2 4

void setup() {
  // This runs ONCE
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
}

void loop() {
  // This runs FOREVER

  digitalWrite(LED1, HIGH);  // LED1 ON
  digitalWrite(LED2, LOW);   // LED2 OFF
  delay(1000);

  digitalWrite(LED1, LOW);   // LED1 OFF
  digitalWrite(LED2, HIGH);  // LED2 ON
  delay(1000);
}
```

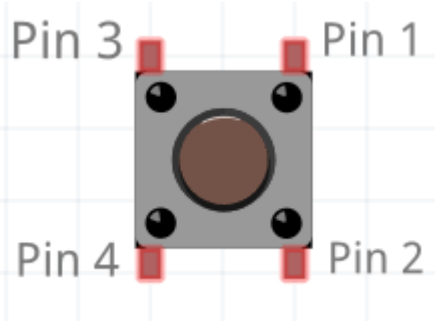
ESP32

```
┌──────────────┐
│              │
│ GPIO 2 ──┼──[220Ω]──▶|── GND   (LED 1)
│              │
│ GPIO 4 ──┼──[220Ω]──▶|── GND   (LED 2)
│              │
│ GND ─────┼──────────────── GND
│              │
└──────────────┘
```

Basic Setup (Blinking Two LEDs)
Hardware: ESP32, Breadboard, Jumper Wires, 2 LEDs, 2 Resistors (220Ω - 330Ω).
Connections:
LED 1: ESP32 GPIO 2 -> Resistor -> LED Anode (+), LED Cathode (-) -> ESP32 GND.
LED 2: ESP32 GPIO 4 -> Resistor -> LED Anode (+), LED Cathode (-) -> ESP32 GND.

```
int ledPin = 5;    // LED connected to digital pin 9
void setup()
 {  // nothing happens in setup}
void loop() {  // fade in from min to max in increments of 5 points:

for (int fadeValue = 0 ; fadeValue <= 255; fadeValue += 5) {    // sets the value (range from 0 to 255):
analogWrite(ledPin, fadeValue);    // wait for 30 milliseconds to see the dimming effect
delay(30);  }  // fade out from max to min in increments of 5 points:

for (int fadeValue = 255 ; fadeValue >= 0; fadeValue -= 5)
{    // sets the value (range from 0 to 255):
analogWrite(ledPin, fadeValue);    // wait for 30 milliseconds to see the dimming effect
 delay(30);
 }
}
```

```
void setup() {
pinMode(5, OUTPUT);
Serial.begin(195200);
while (!Serial);
Serial.println("Input 1 to Turn LED on and 2 to off");}
void loop(){
  if (Serial.available())
 {    int state = Serial.parseInt();
 if (state == 1)    {
 digitalWrite(5, HIGH);
  Serial.println("Command completed LED turned ON");    }
  if (state == 2)    {
 digitalWrite(5, LOW);
 Serial.println("Command completed LED turned OFF");    }    }}
```



Pin1 ──┐
└─ connected inside ── Pin2

Pin3 ──┐
└─ connected inside ── Pin4

When you **press the button**, the two pairs connect, closing the circuit between Pin1/Pin2 and Pin3/Pin4.

You only need to use **one diagonal pair** to make a working switch.

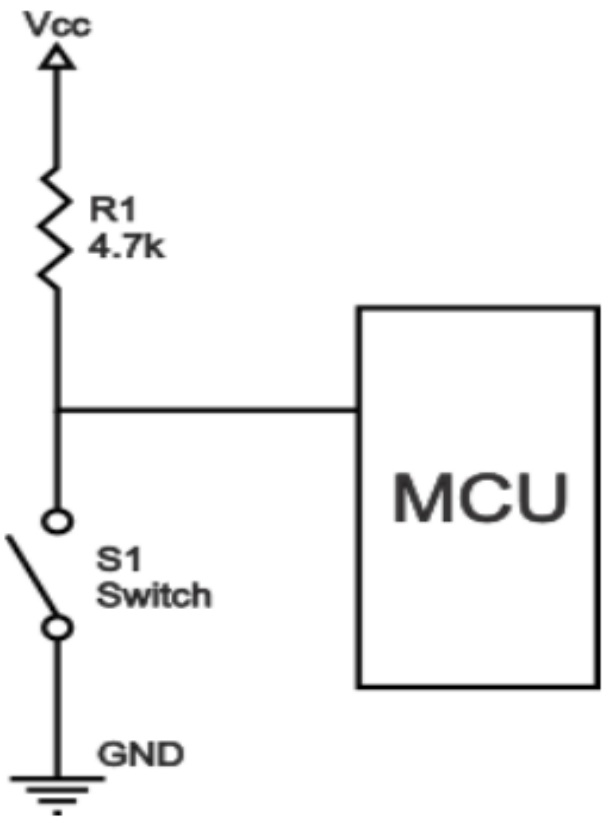
Pull-up Resistor

A pull-up resistor connects an input pin to the positive supply voltage (Vcc or 3.3V/5V).

Default State (Switch Open): The input pin is held at a reliable HIGH logic level due to the connection to the supply voltage.

Active State (Switch Closed): The switch connects the input pin directly to the ground (0V), overriding the pull-up resistor and making the input LOW.

Function: It "pulls" the voltage level up to a high state by default. The circuit in your image is a pull-up configuration for the push button



Pull-up resistor circuit

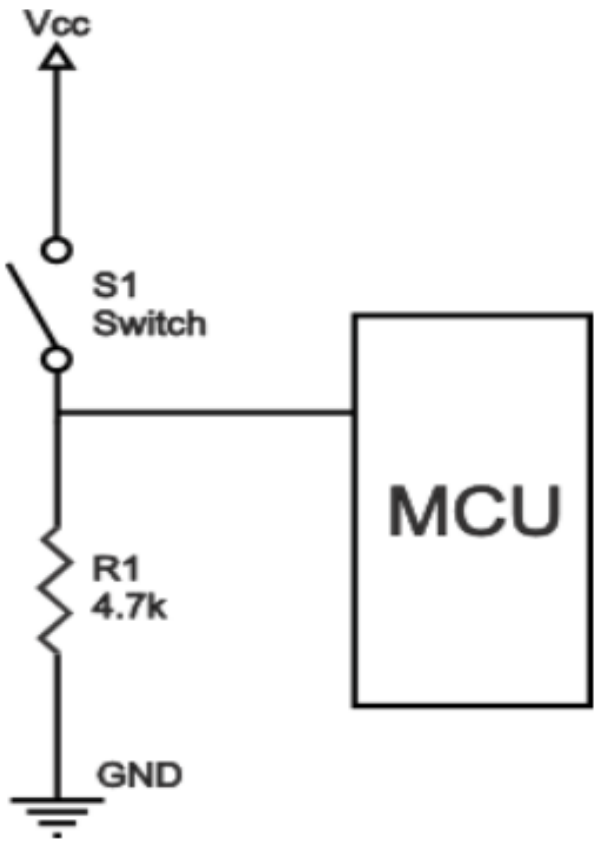
Pull-down Resistor

A pull-down resistor connects the input pin to the ground (0V).

Default State (Switch Open): The input pin is held at a reliable LOW logic level due to the connection to the ground.

Active State (Switch Closed): The switch connects the input pin directly to the positive supply voltage, overriding the pull-down resistor and making the input HIGH.

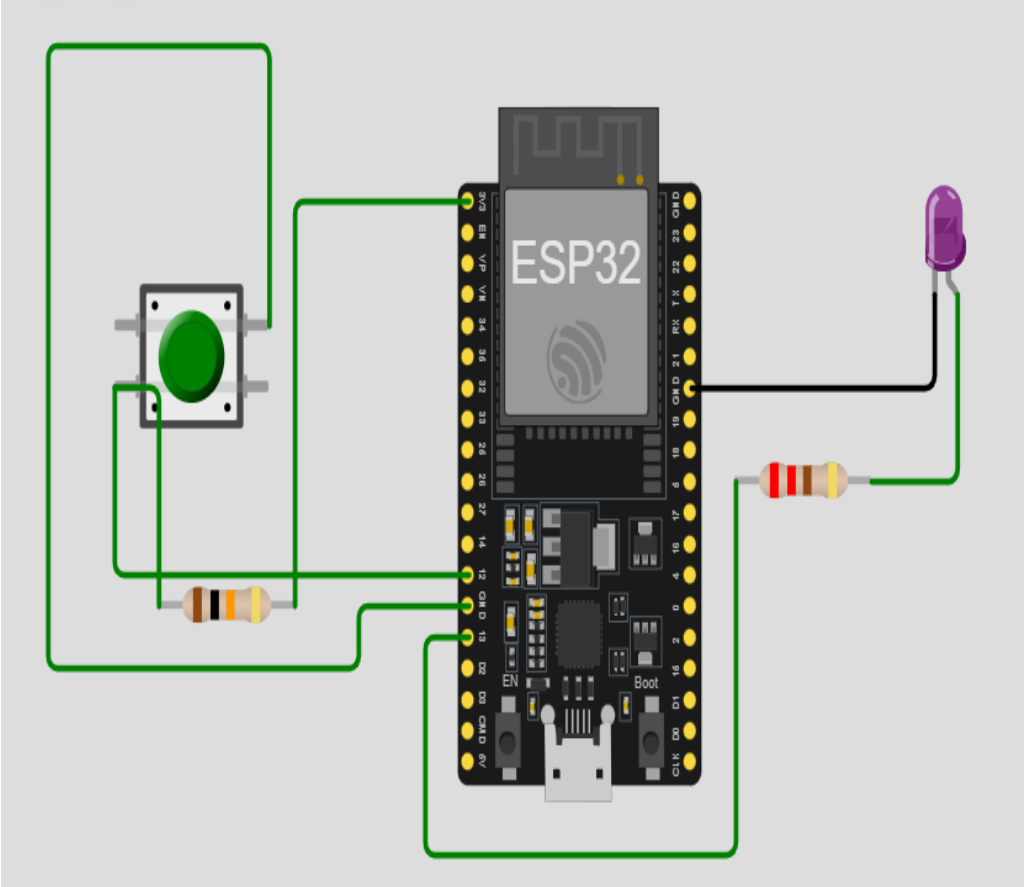
Function: It "pulls" the voltage level down to a low state by default



Pull-down resistor

Using switches and buttons(Pull Up)

```
const int buttonPin = 12;  // the number of the pushbutton pin
const int ledPin =  13;    // the number of the LED pin

// variable for storing the pushbutton status
int buttonState = 0;
void setup() {
  Serial.begin(115200);
  // initialize the pushbutton pin as an input
  pinMode(buttonPin, INPUT_PULLUP);
  // initialize the LED pin as an output
  pinMode(ledPin, OUTPUT);
}
```

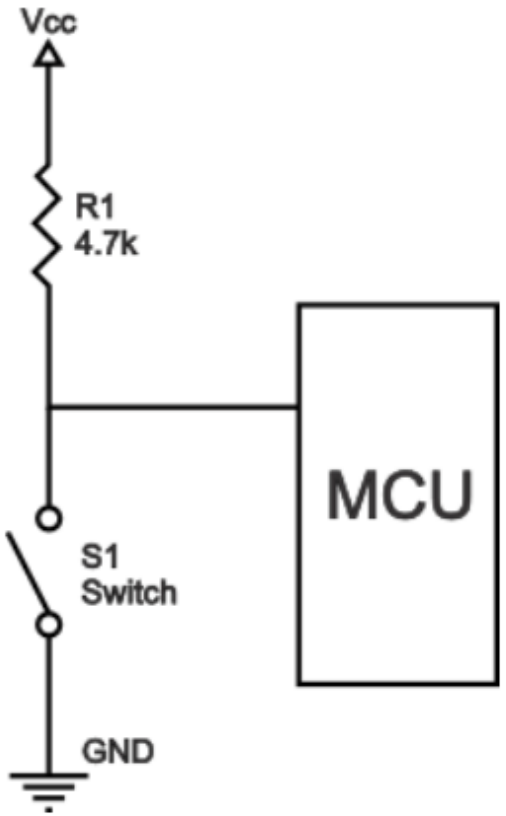


```
void loop() {
  // read the state of the pushbutton value
  buttonState = digitalRead(buttonPin);
  // check if the pushbutton is pressed.
  // if it is, the buttonState is LOW
  if (buttonState == LOW) {
    // turn LED on
    digitalWrite(ledPin, HIGH);
  } else {
    // turn LED off
    digitalWrite(ledPin, LOW);
  }
}
```

// Using INPUT_PULLUP in code eliminates the need for an external resistor.

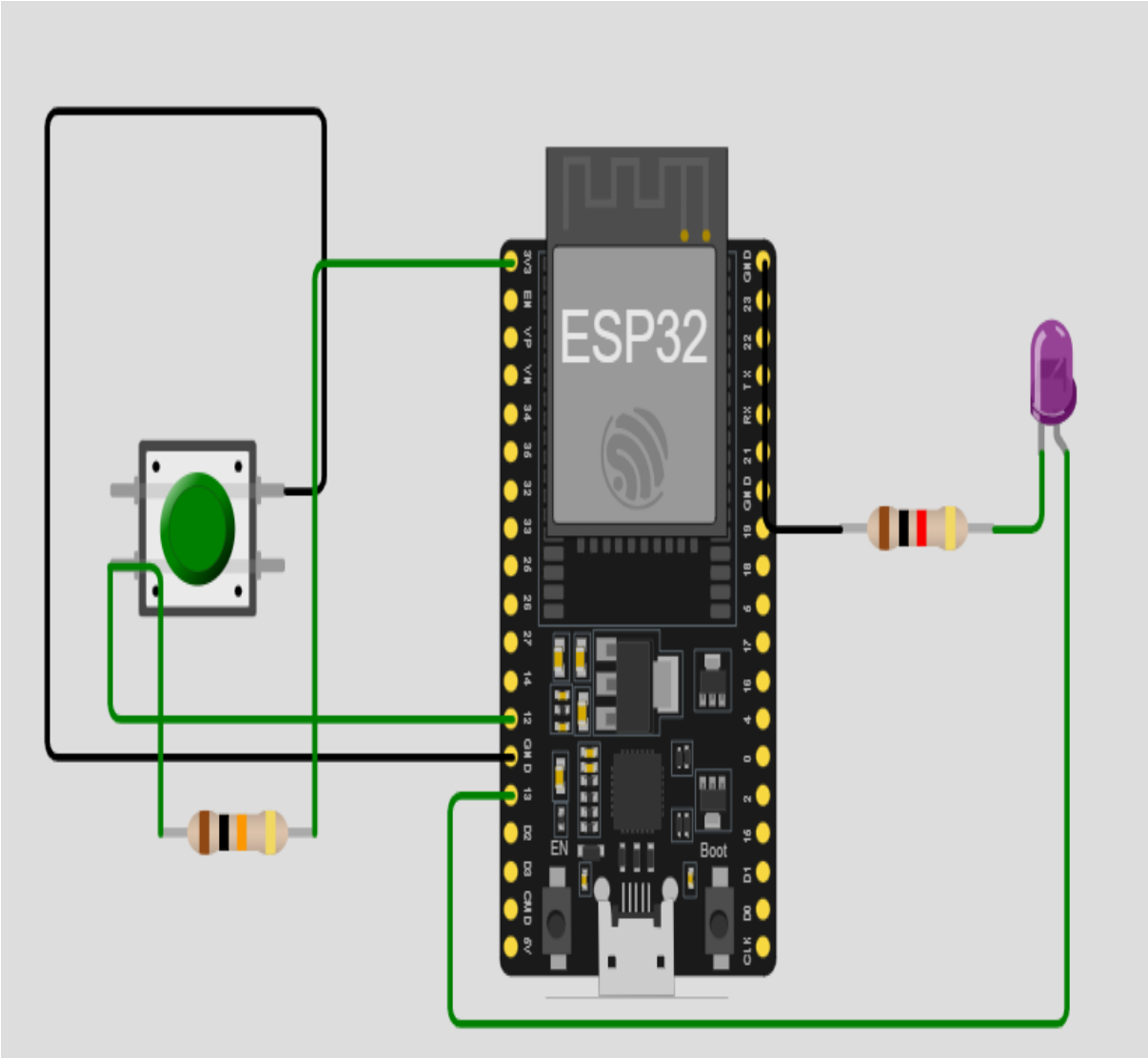
Using switches and buttons(Pull Up) *using resistor.*

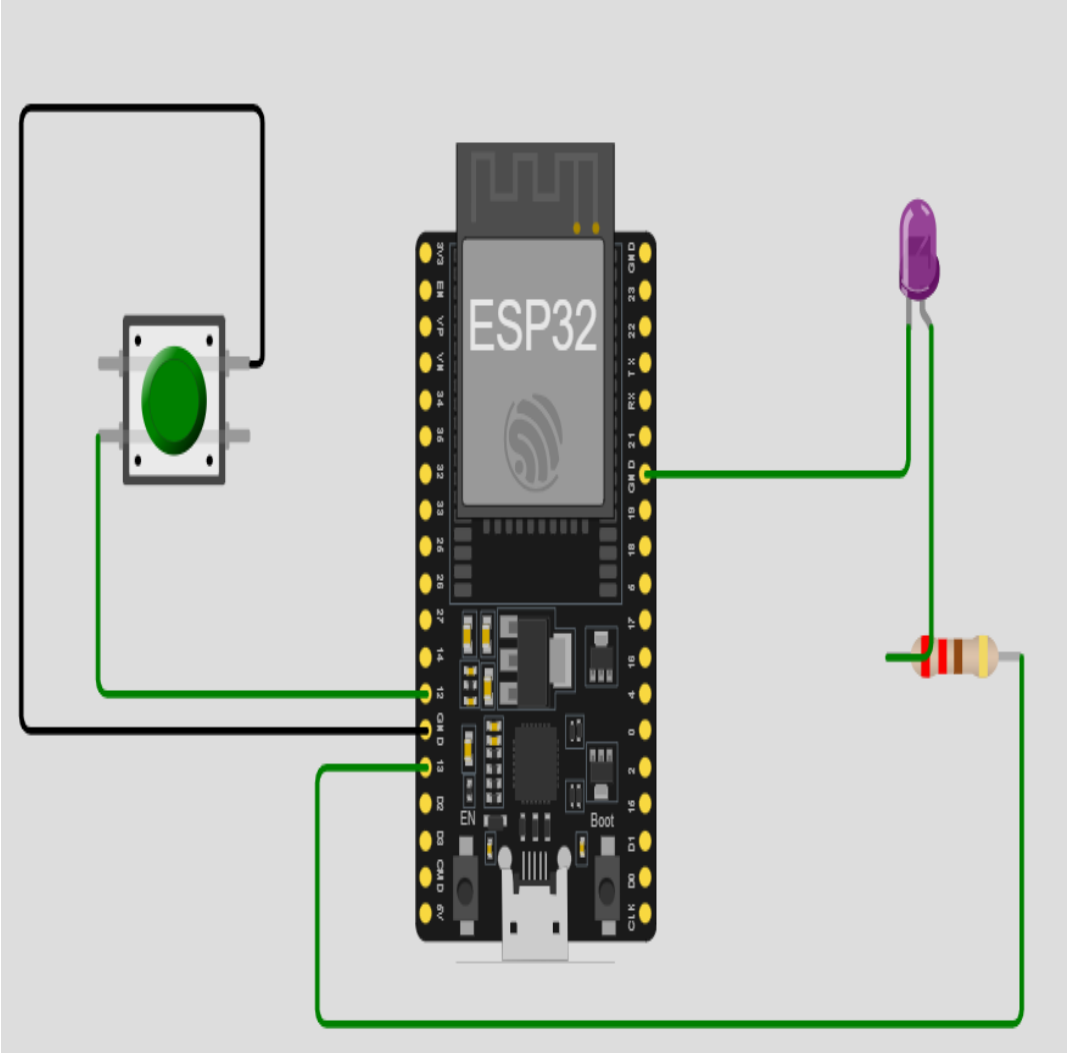
```
#define LED_PIN 13
#define BUTTON_PIN 12
void setup() {
  Serial.begin(115200);
  Serial.println("Hello, ESP32!");
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT);
}
```

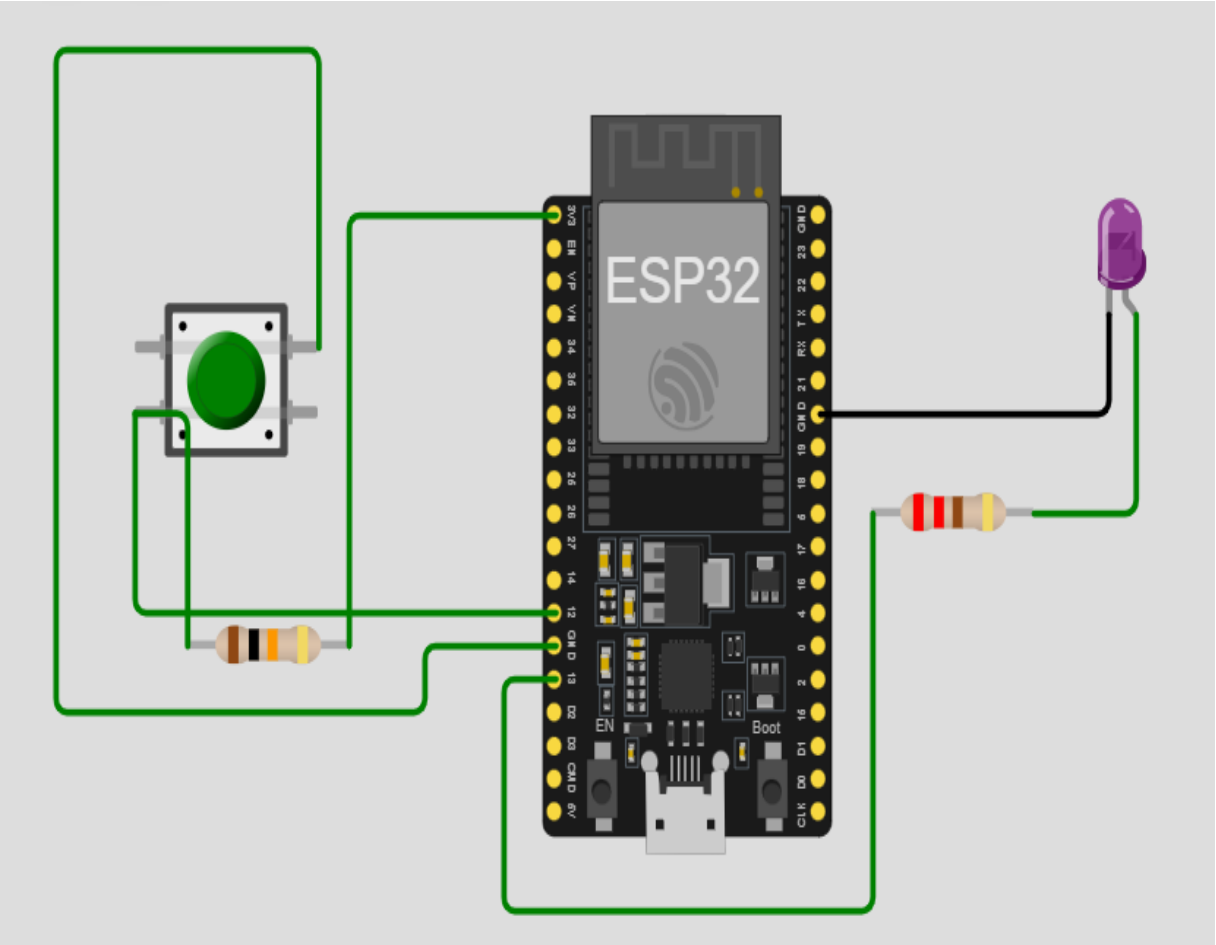


Pull-up resistor circuit

```
void loop() {
  int buttonState = digitalRead(BUTTON_PIN);
  if (buttonState == LOW) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("LED ON");
  } else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("LED OFF");
  }
}
// In this configuration:
  // Button released = HIGH (3.3V via internal resistor)
  // Button pressed = LOW (connected directly to GND)
```

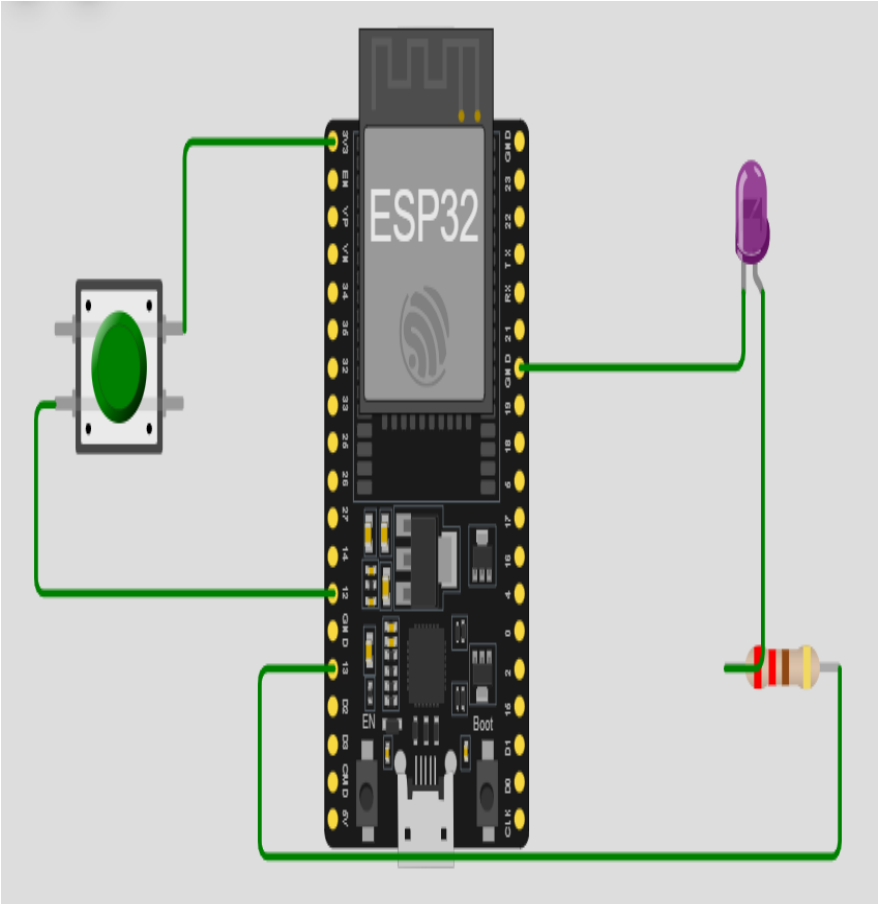



ESP32
EN
Boot


ESP32
EN
Boot

Using switches and buttons(Pull Down with out resistor)

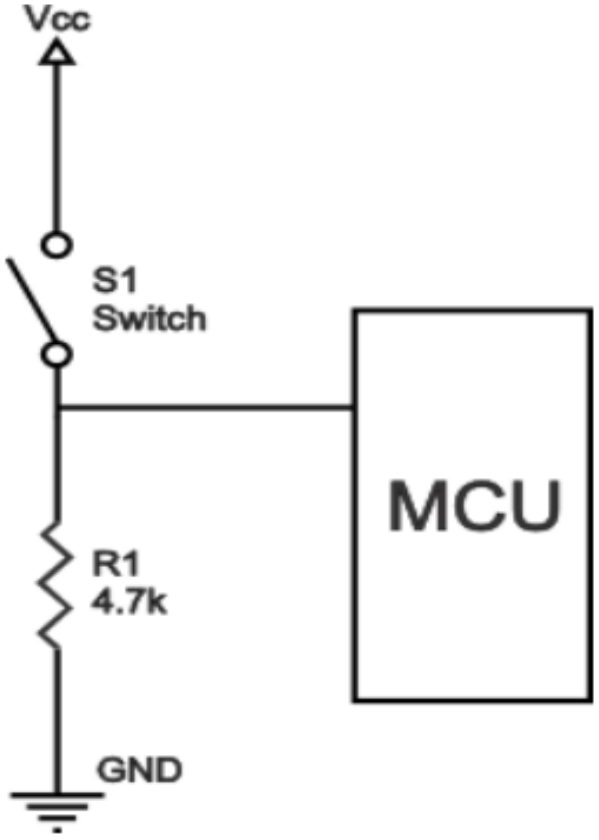
```
#define LED_PIN 13
#define BUTTON_PIN 12
void setup() {
  Serial.begin(115200);
  Serial.println("Hello, ESP32!");
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLDOWN);
}
```



```
void loop() {
  int buttonState = digitalRead(BUTTON_PIN);
// check if the pushbutton is pressed.
  if (buttonState == HIGH) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("LED ON");
  }
else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("LED OFF");
  }
}
```

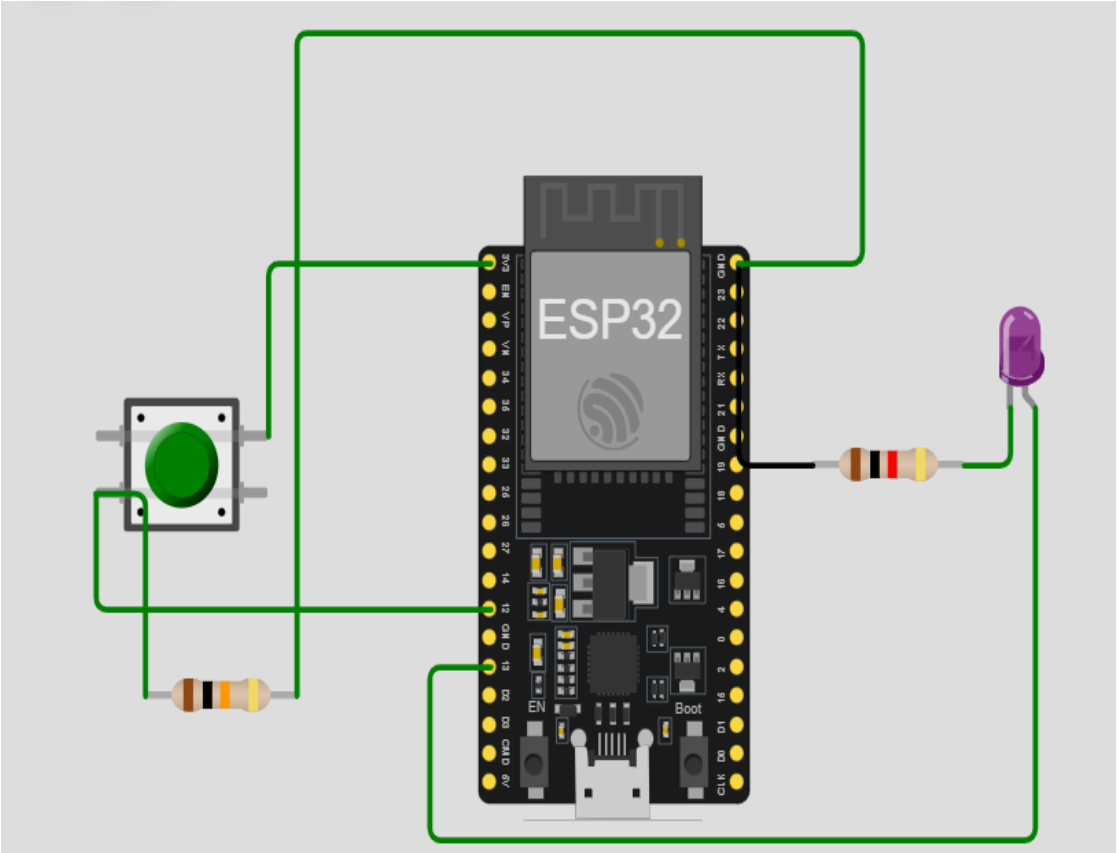
Using switches and buttons(Pull Down)

```
#define LED_PIN 13

#define BUTTON_PIN 12


void setup() {

 Serial.begin(115200);

 Serial.println("Hello, ESP32!");

pinMode(LED_PIN, OUTPUT);

  pinMode(BUTTON_PIN, INPUT);

}
```



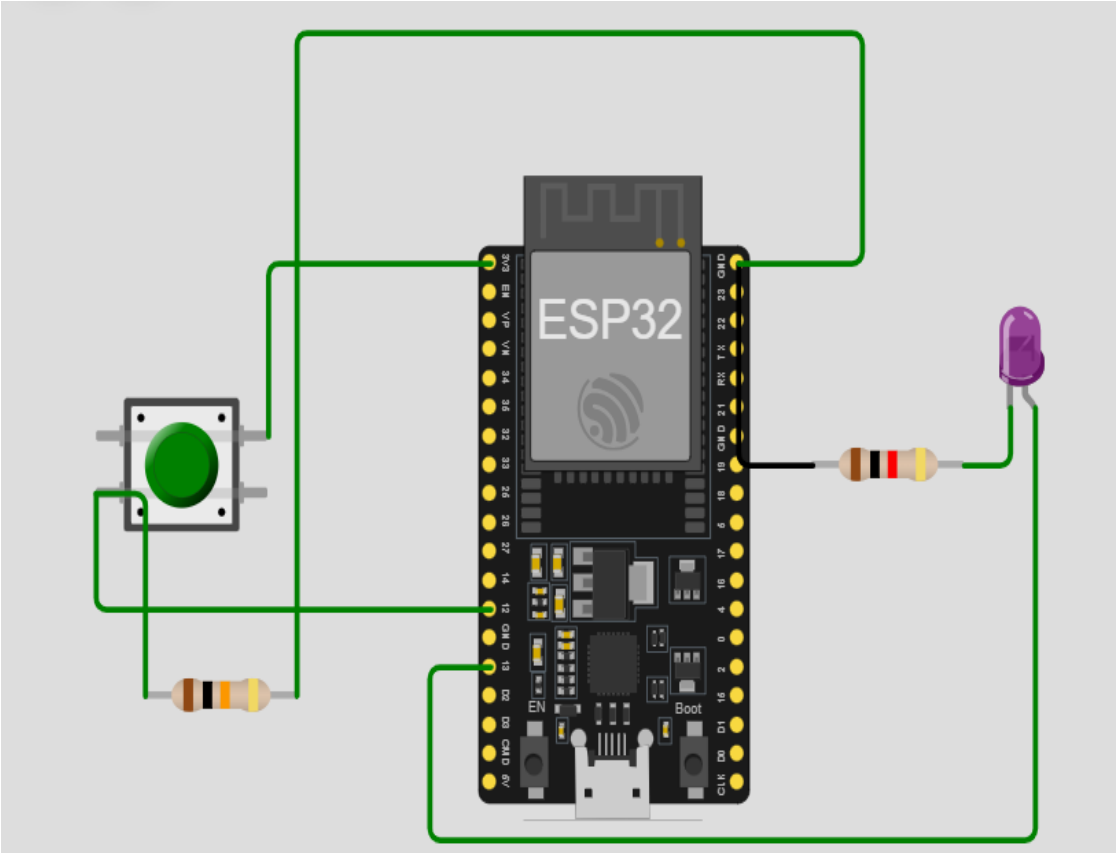
Pull-down resistor

```
void loop() {
  int buttonState = digitalRead(BUTTON_PIN);
//Control LED: If pressed (HIGH), turn ON; else OFF(Pull down)
  if (buttonState == HIGH) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("LED ON");
  } else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("LED OFF");
  }
}
```



//**Button Released:** The serial monitor displays "0" (LOW), and the LED remains OFF.
//Button Pressed: The serial monitor displays "1" (HIGH), and the LED turns ON.

Button not pressed → 10 kΩ pulls GPIO LOW

Button pressed → GPIO gets 3.3 V (HIGH)

LED turns ON only when button is pressed